MCA 1st Semester Lab Manual

DATA STRUCTURS

1. **Given {4,7,3,2,1,7,9,0, find the location of 7 using Binary search and also display its first occurrence.**

```
#include <stdio.h>
#include <stdlib.h>

// Function to sort array using bubble sort
void sortArray(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n-1; i++) {
        for (j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

// Function for binary search - returns first occurrence
int binarySearchFirst(int arr[], int n, int target, int *operations) {
    int left = 0, right = n - 1;
    int firstOccurrence = -1;
    *operations = 0;

    while (left <= right) {
        (*operations)++;
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            firstOccurrence = mid;
            right = mid - 1;  // Continue searching left for first occurrence
        }
        else if (arr[mid] < target) {
            left = mid + 1;
        }
        else {
            right = mid - 1;
```

```
        }
    }
    return firstOccurrence;
}

int main() {
    int arr[] = {4, 7, 3, 2, 1, 7, 9, 0};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 7;
    int sortedArr[8], operations;

    // Copy original array to sort
    for (int i = 0; i < n; i++) {
        sortedArr[i] = arr[i];
    }

    sortArray(sortedArr, n);

    printf("Original array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", sortedArr[i]);
    }
    printf("\n");

    int index = binarySearchFirst(sortedArr, n, target, &operations);

    if (index != -1) {
        printf("First occurrence of %d is at index: %d\n", target, index);
        printf("Value at index %d is: %d\n", index, sortedArr[index]);
    } else {
        printf("%d not found in the array\n", target);
    }
    printf("Number of basic operations (comparisons): %d\n", operations);
    printf("\nTime Complexity: O(log n)\n");

    return 0;
}
```

2. **Given {5,3,1,6,0,2,4} order the numbers in ascending order using Quick Sort.**

```
#include <stdio.h>
int quickSortOperations = 0;

void swap(int *a, int *b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}

int partition(int arr[], int low, int high) {
   int pivot = arr[high];
   int i = (low - 1);

   for (int j = low; j < high; j++) {
      quickSortOperations++;
      if (arr[j] <= pivot) {
         i++;
         swap(&arr[i], &arr[j]);
      }
   }
   swap(&arr[i + 1], &arr[high]);
   return (i + 1);
}

void quickSort(int arr[], int low, int high) {
   if (low < high) {
      int pi = partition(arr, low, high);
      quickSort(arr, low, pi - 1);
      quickSort(arr, pi + 1, high);
   }
}

int main() {
   int arr[] = {5, 3, 1, 6, 0, 2, 4};
   int n = sizeof(arr) / sizeof(arr[0]);
   quickSortOperations = 0;

   printf("Original array: ");
   for (int i = 0; i < n; i++) {
      printf("%d ", arr[i]);
```

```
    }
    printf("\n");

    quickSort(arr, 0, n - 1);

    printf("Sorted array (ascending): ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    printf("Number of basic operations (comparisons): %d\n", quickSortOperations);
    printf("\nTime Complexity: O(n log n) average case, O(n²) worst case\n");

    return 0;
}
```

3. Perform the Merge sort on the input {75,8,1,16,48,3,7,0} and display the output in descending order.

```
#include <stdio.h>

int mergeSortOperations = 0;

void mergeDescending(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;

    while (i < n1 && j < n2) {
        mergeSortOperations++;
        if (L[i] >= R[j]) {  // >= for descending order
            arr[k] = L[i];
```

```
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSortDescending(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSortDescending(arr, left, mid);
        mergeSortDescending(arr, mid + 1, right);
        mergeDescending(arr, left, mid, right);
    }
}

int main() {
    int arr[] = {75, 8, 1, 16, 48, 3, 7, 0};
    int n = sizeof(arr) / sizeof(arr[0]);
    mergeSortOperations = 0;

    printf("Original array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    mergeSortDescending(arr, 0, n - 1);
```

```
    printf("Sorted array (descending): ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    printf("Number of basic operations (comparisons): %d\n", mergeSortOperations);
    printf("\nTime Complexity: O(n log n)\n");

    return 0;
}
```

4. **Write a program to insert the elements 6,1,16,8,27 into singly linked list and delete 8,61,27.**

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

int deleteNode(struct Node** head, int key) {
    if (*head == NULL) {
```

```
        return 0;
    }
    if ((*head)->data == key) {
        struct Node* temp = *head;
        *head = (*head)->next;
        free(temp);
        return 1;
    }

    struct Node* temp = *head;
    while (temp->next != NULL && temp->next->data != key) {
        temp = temp->next;
    }

    if (temp->next != NULL && temp->next->data == key) {
        struct Node* toDelete = temp->next;
        temp->next = temp->next->next;
        free(toDelete);
        return 1;
    }
    return 0;
}

void displayList(struct Node* head) {
    if (head == NULL) {
        printf("Empty");
        return;
    }
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) {
            printf(" -> ");
        }
        temp = temp->next;
    }
}

int listOperationsCount = 0;

int main() {
    struct Node* head = NULL;
    listOperationsCount = 0;
```

```
    int elements[] = {6, 1, 16, 8, 27};
    int n = sizeof(elements) / sizeof(elements[0]);

    printf("Inserting elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", elements[i]);
        insertAtEnd(&head, elements[i]);
        listOperationsCount++;
    }
    printf("\n");

    printf("List after insertion: ");
    displayList(head);
    printf("\n");

    int toDelete[] = {8, 61, 27};
    int delCount = sizeof(toDelete) / sizeof(toDelete[0]);

    for (int i = 0; i < delCount; i++) {
        listOperationsCount++;
        if (deleteNode(&head, toDelete[i])) {
            printf("Deleted %d\n", toDelete[i]);
        } else {
            printf("%d not found\n", toDelete[i]);
        }
        printf("List: ");
        displayList(head);
        printf("\n");
    }

    printf("Total operations performed: %d\n", listOperationsCount);
    printf("\nTime Complexity: O(n) for insertion/deletion\n");

    return 0;
}
```

5.**Write a program to add $6x^3+10x^2+0x+5$ and $4x^2+2x+1$ using linked list.**

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>

struct Term {
```

```
    int coeff;
    int exp;
    struct Term* next;
};

struct Term* createTerm(int coeff, int exp) {
    struct Term* newTerm = (struct Term*)malloc(sizeof(struct Term));
    newTerm->coeff = coeff;
    newTerm->exp = exp;
    newTerm->next = NULL;
    return newTerm;
}

void insertTerm(struct Term** head, int coeff, int exp) {
    if (coeff == 0) return;

    struct Term* newTerm = createTerm(coeff, exp);

    if (*head == NULL || (*head)->exp < exp) {
        newTerm->next = *head;
        *head = newTerm;
        return;
    }

    struct Term* temp = *head;
    while (temp->next != NULL && temp->next->exp > exp) {
        temp = temp->next;
    }
```

```
    if (temp->exp == exp) {
        temp->coeff += coeff;
        free(newTerm);
    } else {
        newTerm->next = temp->next;
        temp->next = newTerm;
    }
}

void displayPolynomial(struct Term* head) {
    if (head == NULL) {
        printf("0");
        return;
    }

    struct Term* temp = head;
    int first = 1;

    while (temp != NULL) {
        if (!first) {
            if (temp->coeff > 0) {
                printf(" + ");
            } else {
                printf(" - ");
            }
        }
        first = 0;

        int absCoeff = abs(temp->coeff);
```

```
        if (temp->exp == 0) {
            printf("%d", absCoeff);
        } else if (temp->exp == 1) {
            printf("%dx", absCoeff);
        } else {
            printf("%dx^%d", absCoeff, temp->exp);
        }
        temp = temp->next;
    }
    printf("\n");
}

struct Term* addPolynomials(struct Term* p1, struct Term* p2) {
    struct Term* result = NULL;
    struct Term* temp1 = p1;
    struct Term* temp2 = p2;

    while (temp1 != NULL && temp2 != NULL) {
        if (temp1->exp == temp2->exp) {
            insertTerm(&result, temp1->coeff + temp2->coeff, temp1->exp);
            temp1 = temp1->next;
            temp2 = temp2->next;
        } else if (temp1->exp > temp2->exp) {
            insertTerm(&result, temp1->coeff, temp1->exp);
            temp1 = temp1->next;
        } else {
            insertTerm(&result, temp2->coeff, temp2->exp);
            temp2 = temp2->next;
        }
```

```
    }

    while (temp1 != NULL) {
        insertTerm(&result, temp1->coeff, temp1->exp);
        temp1 = temp1->next;
    }

    while (temp2 != NULL) {
        insertTerm(&result, temp2->coeff, temp2->exp);
        temp2 = temp2->next;
    }

    return result;
}

int main() {
    struct Term* p1 = NULL;
    struct Term* p2 = NULL;

    // 6x³+10x²+0x+5
    insertTerm(&p1, 6, 3);
    insertTerm(&p1, 10, 2);
    insertTerm(&p1, 0, 1);
    insertTerm(&p1, 5, 0);

    // 4x²+2x+1
    insertTerm(&p2, 4, 2);
    insertTerm(&p2, 2, 1);
    insertTerm(&p2, 1, 0);
```

```
    printf("Polynomial 1: ");
    displayPolynomial(p1);
    printf("Polynomial 2: ");
    displayPolynomial(p2);

    struct Term* result = addPolynomials(p1, p2);

    printf("Sum: ");
    displayPolynomial(result);
    printf("\nTime Complexity: O(m+n) where m,n are number of terms\n");

    return 0;
}
```

6.Write a program to push 5,9,34,17,32 into stack and pop 3 times from the stack, also display the popped numbers.

```
#include <stdio.h>
#define MAX 100
struct Stack {
    int arr[MAX];
    int top;
    int popped[MAX];
    int poppedCount;
    int operations;
};

void initStack(struct Stack* s) {
    s->top = -1;
```

```
    s->poppedCount = 0;
    s->operations = 0;
}

void push(struct Stack* s, int value) {
    if (s->top >= MAX - 1) {
        printf("Stack overflow!\n");
        return;
    }
    s->operations++;
    s->arr[++(s->top)] = value;
    printf("Pushed: %d\n", value);
}

int pop(struct Stack* s) {
    if (s->top < 0) {
        printf("Stack underflow!\n");
        return -1;
    }
    s->operations++;
    int value = s->arr[(s->top)--];
    s->popped[s->poppedCount++] = value;
    printf("Popped: %d\n", value);
    return value;
}

void displayStack(struct Stack* s) {
    if (s->top < 0) {
        printf("Empty");
```

```
        return;
    }
    for (int i = 0; i <= s->top; i++) {
        printf("%d", s->arr[i]);
        if (i < s->top) printf(", ");
    }
}

int main() {
    struct Stack s;
    initStack(&s);

    int elements[] = {5, 9, 34, 17, 32};
    int n = sizeof(elements) / sizeof(elements[0]);

    printf("Pushing elements:\n");
    for (int i = 0; i < n; i++) {
        push(&s, elements[i]);
    }

    printf("\nStack after pushes: [");
    displayStack(&s);
    printf("]\n");

    printf("\nPopping 3 elements:\n");
    for (int i = 0; i < 3; i++) {
        pop(&s);
    }
```

```
    printf("\nPopped elements: ");
    for (int i = 0; i < s.poppedCount; i++) {
        printf("%d", s.popped[i]);
        if (i < s.poppedCount - 1) printf(", ");
    }
    printf("\n");

    printf("Remaining stack: [");
    displayStack(&s);
    printf("]\n");

    printf("Total operations performed: %d\n", s.operations);
    printf("\nTime Complexity: O(1) per push/pop operation\n");

    return 0;
}
```

7. Write a recursive program to find GCD of 4,6,8.

```
#include <stdio.h>

int gcdRecursive(int a, int b, int* operations) {
    (*operations)++;
    if (b == 0) {
        return a;
    }
    return gcdRecursive(b, a % b, operations);
}

int gcdOfArray(int arr[], int n, int* operations) {
    *operations = 0;
```

```
    int result = arr[0];
    for (int i = 1; i < n; i++) {
        result = gcdRecursive(result, arr[i], operations);
    }
    return result;
}

int main() {
    int arr[] = {4, 6, 8};
    int n = sizeof(arr) / sizeof(arr[0]);
    int operations = 0;

    printf("Numbers: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int result = gcdOfArray(arr, n, &operations);

    printf("GCD: %d\n", result);
    printf("Number of recursive calls: %d\n", operations);
    printf("\nTime Complexity: O(log min(a,b))\n");

    return 0;
}
```

8. Write a program to insert the elements {5,7,0,6,3,9} into circular queue and delete 6,9&5 from it (using linked list implementation).

```
#include <stdio.h>
```

```
#include <stdlib.h>

struct CQNode {
    int data;
    struct CQNode* next;
};

struct CircularQueue {
    struct CQNode* rear;
    int operations;
};

void initCircularQueue(struct CircularQueue* cq) {
    cq->rear = NULL;
    cq->operations = 0;
}

void enqueueCQ(struct CircularQueue* cq, int data) {
    cq->operations++;
    struct CQNode* newNode = (struct CQNode*)malloc(sizeof(struct CQNode));
    newNode->data = data;

    if (cq->rear == NULL) {
        cq->rear = newNode;
        newNode->next = newNode;
    } else {
        newNode->next = cq->rear->next;
        cq->rear->next = newNode;
        cq->rear = newNode;
```

```
    }
}

int dequeueCQ(struct CircularQueue* cq) {
    cq->operations++;
    if (cq->rear == NULL) {
        return -1;
    }

    if (cq->rear->next == cq->rear) {
        int data = cq->rear->data;
        free(cq->rear);
        cq->rear = NULL;
        return data;
    }

    struct CQNode* front = cq->rear->next;
    int data = front->data;
    cq->rear->next = front->next;
    free(front);
    return data;
}

int deleteValueCQ(struct CircularQueue* cq, int value) {
    if (cq->rear == NULL) {
        return 0;
    }

    struct CQNode* current = cq->rear->next;
```

```
    struct CQNode* prev = cq->rear;

    while (1) {
        cq->operations++;
        if (current->data == value) {
            if (current == cq->rear->next) {
                cq->rear->next = current->next;
            } else if (current == cq->rear) {
                cq->rear = prev;
            } else {
                prev->next = current->next;
            }
            free(current);
            return 1;
        }

        prev = current;
        current = current->next;

        if (current == cq->rear->next) {
            break;
        }
    }
    return 0;
}

void displayCQ(struct CircularQueue* cq) {
    if (cq->rear == NULL) {
        printf("Empty");
```

```
        return;
    }

    struct CQNode* current = cq->rear->next;
    int first = 1;
    while (1) {
        if (!first) printf(" -> ");
        printf("%d", current->data);
        first = 0;
        current = current->next;
        if (current == cq->rear->next) break;
    }
    printf(" (circular)");
}

int main() {
    struct CircularQueue cq;
    initCircularQueue(&cq);

    int elements[] = {5, 7, 0, 6, 3, 9};
    int n = sizeof(elements) / sizeof(elements[0]);

    printf("Enqueuing elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", elements[i]);
        enqueueCQ(&cq, elements[i]);
    }
    printf("\n");
```

```
    printf("Queue: ");
    displayCQ(&cq);
    printf("\n");

    int toDelete[] = {6, 9, 5};
    int delCount = sizeof(toDelete) / sizeof(toDelete[0]);

    for (int i = 0; i < delCount; i++) {
        if (deleteValueCQ(&cq, toDelete[i])) {
            printf("Deleted %d\n", toDelete[i]);
        } else {
            printf("%d not found\n", toDelete[i]);
        }
        printf("Queue: ");
        displayCQ(&cq);
        printf("\n");
    }

    printf("Total operations performed: %d\n", cq.operations);
    printf("\nTime Complexity: O(1) for enqueue/dequeue, O(n) for deletion\n");

    return 0;
}
```

9. Given S1={"Flowers"}, S2={"are beautiful"},
a) Find the length of S1.
b) Concatenate S1 and S2.
c) Extract the substring "low" from S1.
d) Find "are" in S2 and replace it with "is".

```
#include <stdio.h>
```

```
#include <string.h>

int main() {
    char S1[] = "Flowers";
    char S2[] = "are beautiful";
    int operations = 0;

    // a) Length of S1
    operations++;
    int len1 = strlen(S1);
    printf("a) Length of S1: %d\n", len1);

    // b) Concatenate S1 and S2
    operations++;
    char concat[100];
    strcpy(concat, S1);
    strcat(concat, " ");
    strcat(concat, S2);
    printf("b) Concatenation: %s\n", concat);

    // c) Extract substring "low" from S1
    operations++;
    char* pos = strstr(S1, "low");
    if (pos != NULL) {
        char substring[4];
        strncpy(substring, pos, 3);
        substring[3] = '\0';
        printf("c) Substring extracted: '%s'\n", substring);
    } else {
```

```
        printf("c) 'low' not found in S1\n");
    }

    // d) Find "are" in S2 and replace with "is"
    operations++;
    char* pos2 = strstr(S2, "are");
    if (pos2 != NULL) {
        char replaced[100];
        int index = pos2 - S2;
        strncpy(replaced, S2, index);
        replaced[index] = '\0';
        strcat(replaced, "is");
        strcat(replaced, S2 + index + 3);
        printf("d) After replacement: %s\n", replaced);
    } else {
        printf("d) 'are' not found in S2\n");
    }

    printf("Total operations: %d\n", operations);
    printf("\nTime Complexity: O(n) for each string operation\n");

    return 0;
}
```

10. Write a program to convert an infix expression x^y/(5*2)+2 to its postfix expression.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAX 100

struct Stack2 {
    char arr[MAX];
    int top;
};

void initStack2(struct Stack2* s) {
    s->top = -1;
}

void push2(struct Stack2* s, char c) {
    if (s->top < MAX - 1) {
        s->arr[++(s->top)] = c;
    }
}

char pop2(struct Stack2* s) {
    if (s->top >= 0) {
        return s->arr[(s->top)--];
    }
    return '\0';
}

char peek2(struct Stack2* s) {
    if (s->top >= 0) {
        return s->arr[s->top];
    }
    return '\0';
```

```
}

int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

void infixToPostfix(char* infix, char* postfix) {
    struct Stack2 s;
    initStack2(&s);
    int j = 0;
    int operations = 0;

    printf("Converting: %s\n", infix);

    for (int i = 0; infix[i] != '\0'; i++) {
        operations++;
        char c = infix[i];

        if (isalnum(c)) {
            postfix[j++] = c;
        } else if (c == '(') {
            push2(&s, c);
        } else if (c == ')') {
            while (s.top >= 0 && peek2(&s) != '(') {
                postfix[j++] = pop2(&s);
            }
```

```
            pop2(&s);  // Remove '('
        } else {
            while (s.top >= 0 && precedence(peek2(&s)) >= precedence(c)) {
                postfix[j++] = pop2(&s);
            }
            push2(&s, c);
        }
    }

    while (s.top >= 0) {
        postfix[j++] = pop2(&s);
    }
    postfix[j] = '\0';

    printf("Number of operations: %d\n", operations);
}

int main() {
    char infix[] = "x^y/(5*2)+2";
    char postfix[MAX];

    infixToPostfix(infix, postfix);

    printf("Infix expression: %s\n", infix);
    printf("Postfix expression: %s\n", postfix);
    printf("\nTime Complexity: O(n) where n is length of expression\n");

    return 0;
}
```

11. Write a program to evaluate a postfix expression 5 3+8 2-*.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX 100

struct Stack3 {
    int arr[MAX];
    int top;
};

void initStack3(struct Stack3* s) {
    s->top = -1;
}

void push3(struct Stack3* s, int val) {
    if (s->top < MAX - 1) {
        s->arr[++(s->top)] = val;
    }
}

int pop3(struct Stack3* s) {
    if (s->top >= 0) {
        return s->arr[(s->top)--];
    }
    return 0;
}
```

```
int evaluatePostfix(char* expression) {
    struct Stack3 s;
    initStack3(&s);
    int operations = 0;

    printf("Evaluating: %s\n", expression);

    for (int i = 0; expression[i] != '\0'; i++) {
        operations++;
        char c = expression[i];

        if (isdigit(c)) {
            push3(&s, c - '0');
        } else if (c == ' ') {
            continue;
        } else {
            int op2 = pop3(&s);
            int op1 = pop3(&s);
            int result;

            switch(c) {
                case '+': result = op1 + op2; break;
                case '-': result = op1 - op2; break;
                case '*': result = op1 * op2; break;
                case '/': result = op1 / op2; break;
                default: result = 0;
            }
            push3(&s, result);
```

```
        }
    }

    printf("Number of operations: %d\n", operations);
    return pop3(&s);
}

int main() {
    char expression[] = "5 3+8 2-*";

    int result = evaluatePostfix(expression);

    printf("Expression: %s\n", expression);
    printf("Result: %d\n", result);
    printf("\nTime Complexity: O(n) where n is length of expression\n");

    return 0;
}
```

12. Write a program to create a binary tree with the elements 18,15,40,50,30,17,41 after creation insert 45 and 19 into tree and delete 15,17 and 41 from tree. Display the tree on each insertion and deletion operation.

```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};
```

```
struct TreeNode* createTreeNode(int data) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct TreeNode* insertBST(struct TreeNode* root, int data) {
    if (root == NULL) {
        return createTreeNode(data);
    }

    if (data < root->data) {
        root->left = insertBST(root->left, data);
    } else if (data > root->data) {
        root->right = insertBST(root->right, data);
    }
    return root;
}

struct TreeNode* minValueNode(struct TreeNode* node) {
    struct TreeNode* current = node;
    while (current && current->left != NULL) {
        current = current->left;
    }
    return current;
}
```

```
struct TreeNode* deleteBST(struct TreeNode* root, int data) {
    if (root == NULL) return root;

    if (data < root->data) {
        root->left = deleteBST(root->left, data);
    } else if (data > root->data) {
        root->right = deleteBST(root->right, data);
    } else {
        if (root->left == NULL) {
            struct TreeNode* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct TreeNode* temp = root->left;
            free(root);
            return temp;
        }

        struct TreeNode* temp = minValueNode(root->right);
        root->data = temp->data;
        root->right = deleteBST(root->right, temp->data);
    }
    return root;
}

void inorderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
```

```
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

void displayTree(struct TreeNode* root) {
    printf("Inorder: ");
    inorderTraversal(root);
    printf("\n");
}

int main() {
    struct TreeNode* root = NULL;

    int elements[] = {18, 15, 40, 50, 30, 17, 41};
    int n = sizeof(elements) / sizeof(elements[0]);

    printf("Creating tree with elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", elements[i]);
        root = insertBST(root, elements[i]);
    }
    printf("\n");

    printf("Tree after creation:\n");
    displayTree(root);

    // Insert 45 and 19
    printf("\nInserting 45 and 19:\n");
```

```
    root = insertBST(root, 45);
    root = insertBST(root, 19);
    displayTree(root);

    // Delete 15, 17, and 41
    printf("\nDeleting 15, 17, and 41:\n");
    root = deleteBST(root, 15);
    printf("After deleting 15: ");
    displayTree(root);

    root = deleteBST(root, 17);
    printf("After deleting 17: ");
    displayTree(root);

    root = deleteBST(root, 41);
    printf("After deleting 41: ");
    displayTree(root);

    printf("\nTime Complexity: O(log n) for insert/delete in balanced tree\n");

    return 0;
}
```

13. Write a program to create binary search tree with the elements {2,5,1,3,9,0,6} and perform inorder, preorder and post order traversal.

```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
```

```
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};

struct TreeNode* createTreeNode(int data) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct TreeNode* insertBST(struct TreeNode* root, int data) {
    if (root == NULL) {
        return createTreeNode(data);
    }

    if (data < root->data) {
        root->left = insertBST(root->left, data);
    } else if (data > root->data) {
        root->right = insertBST(root->right, data);
    }
    return root;
}

void inorderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
```

```
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

void preorderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}

void postorderTraversal(struct TreeNode* root) {
    if (root != NULL) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    struct TreeNode* root = NULL;

    int elements[] = {2, 5, 1, 3, 9, 0, 6};
    int n = sizeof(elements) / sizeof(elements[0]);

    printf("Creating BST with elements: ");
    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", elements[i]);
        root = insertBST(root, elements[i]);
    }
    printf("\n\n");

    printf("Inorder traversal (sorted): ");
    inorderTraversal(root);
    printf("\n");

    printf("Preorder traversal (root, left, right): ");
    preorderTraversal(root);
    printf("\n");

    printf("Postorder traversal (left, right, root): ");
    postorderTraversal(root);
    printf("\n");

    printf("\nTime Complexity: O(n) for each traversal\n");

    return 0;
}
```

14. **Write a program to Sort the following elements using heap sort {9,16,32,8,4,1,5,8,0}.**

```
#include <stdio.h>
int heapSortOperations = 0;

void swap2(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
```

```
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n) {
        heapSortOperations++;
        if (arr[left] > arr[largest]) {
            largest = left;
        }
    }

    if (right < n) {
        heapSortOperations++;
        if (arr[right] > arr[largest]) {
            largest = right;
        }
    }

    if (largest != i) {
        swap2(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
```

```
        heapify(arr, n, i);
    }

    for (int i = n - 1; i > 0; i--) {
        swap2(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int arr[] = {9, 16, 32, 8, 4, 1, 5, 8, 0};
    int n = sizeof(arr) / sizeof(arr[0]);
    heapSortOperations = 0;

    printf("Original array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    heapSort(arr, n);

    printf("Sorted array (ascending): ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    printf("Number of basic operations (comparisons): %d\n", heapSortOperations);
```

```
    printf("\nTime Complexity: O(n log n)\n");

    return 0;
}
```